# Adaptive Path Planning for Autonomous Vehicles in Lane-Free Mixed Traffic: A Modular Perception-to-Control Pipeline

Harish Kumar Rai, Renu Kumar
*Computer Science Department, IIMT Engineering College*
*Meerut, India*
harrishrai701@gmail.com, renukumar_cse@iimtindia.net

***Abstract*—Indian roads often lack lane markings and mix cars, buses, auto-rickshaws, carts, cyclists, pedestrians and cattle, which violates the lane-graph assumptions of most motion planners. We present a closed-loop autonomous driving stack for such lane-free, mixed traffic. It comprises simulated camera, radar and 32-beam LiDAR; LiDAR clustering with L-shape box fitting; an extended Kalman filter tracker with a constant turn rate and velocity motion model and Hungarian data association; class-conditioned multi-hypothesis prediction; a six-state behavior layer that outputs planner parameters rather than controls; a Frenet-frame lattice planner; and pure-pursuit and PI control of a kinematic bicycle model. The key planning choice is to parameterize lateral motion by arc length rather than time, which keeps every candidate within the vehicle's steering limits at low speed. Safety is measured against ground truth rather than the vehicle's own tracks. Across five scenarios—an unmarked village road, an unsignalized crossroads, a highway merge, a dense market street and a sudden cattle crossing—the vehicle reaches its goal in all cases with zero collisions, minimum clearance of 0.28–3.89 m and 95th-percentile replanning latency of 30–37 ms in an interpreter. Over 15 randomized worlds it records zero collisions and 14 arrivals. The implementation is portable MATLAB/GNU Octave code without toolbox dependencies, with a Unity 6 front end that runs the same stack against rendered sensors.**

***Index Terms*—autonomous vehicles, motion planning, Frenet frame, lattice planner, sensor fusion, extended Kalman filter, behavior planning, unstructured traffic.**

## I. Introduction

Most motion planners for automated driving assume structure that many Indian roads lack. Lane graphs, right-of-way rules and a small set of road-user classes suit a marked highway, but they fail on an unmarked village road, a market street narrowed by stalls, or a rural road where cattle step out from behind a parked truck [14].

On such roads the vehicle must choose its own lateral position, share space with slow and unpredictable agents, and stop only for obstacles that are really present. A planner that stops for every false detection never arrives; one that trusts every track drives into whatever its tracker has lost.

This paper describes a complete perception-to-control stack designed for these conditions and evaluates it in closed loop. Its contributions are:

1) a lane-free Frenet lattice planner whose lateral motion is parameterized by arc length rather than time, keeping candidates within the steering limits at low speed;
2) a behavior layer that outputs planner parameters—a speed cap, a lateral limit and a risk tolerance—rather than control commands;
3) footprint-based tracking, in which each track carries a center, long-axis orientation and size learned from L-shape LiDAR box fits;
4) evidence-gated caution, under which an unconfirmed object can cap the speed but never trigger a stop; and
5) ground-truth safety evaluation with a portable, reproducible MATLAB/GNU Octave implementation and a Unity 6 co-simulation front end.

Section II reviews related work. Section III presents the architecture and Section IV the methods. Section V describes the implementation and Section VI the results. Sections VII and VIII discuss limitations and conclude.

## II. Related Work

*Motion planning.* Paden *et al.* survey planning and control for self-driving vehicles [1]. Werling *et al.* generate optimal trajectories in a Frenét frame by sampling quintic lateral and quartic longitudinal polynomials in time and selecting by cost [2]. State-lattice planners sample a discretized set of dynamically feasible motions [3], and Hybrid A* searches a continuous-state grid for unstructured environments [4]. We retain the polynomial lattice of [2] but parameterize lateral motion by arc length, and keep Hybrid A* only as a disabled fallback.

*Behavior.* Vehicles in the DARPA Urban Challenge selected driving behavior with finite-state machines [5]. Our state machine differs in that it emits parameters to a single optimizer instead of switching between controllers.

*Tracking and fusion.* The extended Kalman filter (EKF) is the standard estimator for nonlinear target motion [6], and the constant turn rate and velocity (CTRV) model offers a good accuracy–complexity trade-off for vehicles [7]. We associate detections with the Hungarian algorithm [8] behind a $\chi^2$ gate and confirm tracks with M-of-N logic [9]. L-shape fitting recovers vehicle boxes from the partial outline seen by a LiDAR [10].

*Prediction.* Lefèvre *et al.* classify motion prediction into physics-based, maneuver-based and interaction-aware approaches [11]. Our predictor is maneuver-based, with a small set of class-conditioned hypotheses per agent.

*Control and Indian road conditions.* We track the plan with pure pursuit [12] on a kinematic bicycle model, which is adequate for planning-level control at moderate lateral acceleration [13]. The India Driving Dataset (IDD) documents the unstructured scenes and long-tailed classes of Indian roads [14]; our scenarios are built around the situations it describes.

## III. System Architecture

Fig. 1 shows the stack as a single closed loop. The simulation advances at 20 Hz ($\Delta t$ = 0.05 s). Sensing, perception, tracking and control run every step; risk assessment, behavior selection, prediction and planning run every 0.1 s. Tracks feed two parallel branches: risk assessment and a behavior state machine convert them into limits for the planner, while the predictor converts them into weighted future footprints. The lattice planner combines both, and the controller tracks the selected trajectory.

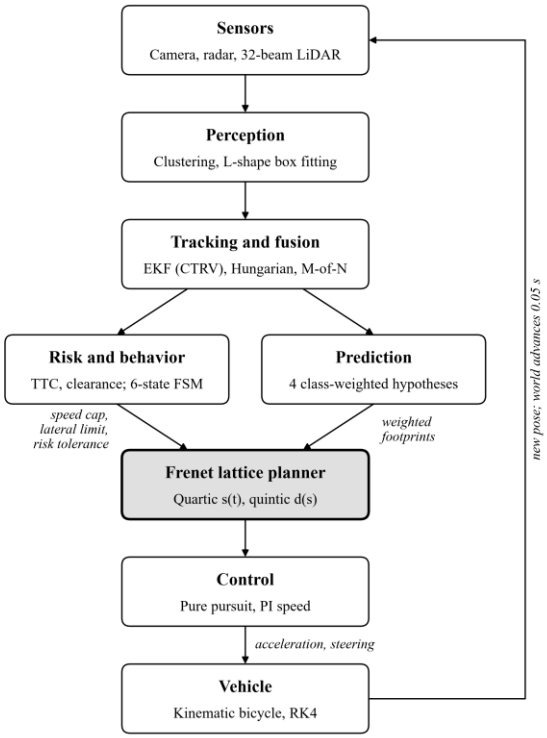



Fig. 1. System architecture. The behavior layer only bounds the planner (shaded); it never commands the vehicle directly.

The single exception to that rule is a reverse maneuver that overrides the planner when the vehicle is boxed in (Section IV-D). The same loop runs in two hosts. Offline, an Octave world model supplies sensor data and integrates the vehicle; live, Unity renders the sensors and moves the vehicle while the Octave stack serves commands over a socket.

## IV. Methods

### A. Sensing and Perception

Three sensors are simulated with ray-cast occlusion, per-sensor detection probability $P_D$ and clutter at a mean of 0.6 false returns per step (Table I). The camera labels classes with 88% accuracy, the radar also measures range rate ($\sigma$ = 0.10 m/s), and the LiDAR has 32 beams over ±15° with 1024 azimuth bins and 5% dropout.

TABLE I
Sensor Models

| Sensor | Range (m) | FOV | $P_D$ | Noise (1$\sigma$) |
|---|---|---|---|---|
| Camera | 60 | 100° | 0.92 | 1.5 px |
| Radar | 90 | 60° | 0.90 | 0.25 m, 2° |
| LiDAR | 40 | 360° | 0.95 | 0.02 m |

LiDAR returns 0.25–4.0 m above ground inside a region of interest are binned into 0.3 m cells and grouped into connected components. Each cluster is fitted with an oriented box by L-shape fitting [10]: 30 candidate orientations over a quarter turn are scored by the proximity of points to the box edges, with a 15% bonus for orientations aligned with the road. A principal-axis fit was rejected because the L-shaped outline that a LiDAR observes of a car makes its principal axis the diagonal, which rotates as the car passes. Each box becomes a position measurement whose covariance grows with the box size.

### B. Tracking and Fusion

Each track carries the CTRV state $\mathbf{x} = [p_x, p_y, v, \psi, \omega]^T$ [7]. For $\omega \neq 0$ the position propagates as

$$p'_x = p_x + \frac{v}{\omega}(\sin(\psi + \omega\Delta t) - \sin\psi) \quad (1)$$

$$p'_y = p_y + \frac{v}{\omega}(\cos\psi - \cos(\psi + \omega\Delta t)) \quad (2)$$

with $\psi' = \psi + \omega\Delta t$ and $v$, $\omega$ held constant; a second-order expansion is used for $|\omega| < 10^{-4}$. The EKF [6] linearizes the model with its analytic Jacobian and adds process noise from longitudinal acceleration ($\sigma$ = 1.5 m/s$^2$) and yaw acceleration ($\sigma$ = 0.6 rad/s$^2$). The update uses position measurements and the Joseph-form covariance update for numerical stability.

Detections are associated with tracks per sensor by the Hungarian algorithm [8], gated by a normalized innovation squared below 9.21, the 99% $\chi^2$ bound for two degrees of freedom. Before association, several LiDAR fragments lying inside one settled track, such as the side of a bus seen at a grazing angle, are merged into one box. A track is confirmed after three hits in four steps [9], and only if a LiDAR, or both camera and radar, have observed it. It becomes *established* after 12 hits (0.6 s) with a recent LiDAR return when within LiDAR range, and only established tracks can cause a stop. Tracks are deleted after 6 misses, or 20 for stationary objects, so that parked vehicles survive brief occlusion.

Each track also learns a *footprint*—center, long-axis orientation and size—by exponential smoothing of its LiDAR boxes. Because a box defines orientation only modulo a quarter turn, the established axis, the direction of travel or the road heading resolves which side is long. A track is declared still after 1.0 s below 0.5 m/s and moving after 0.5 s above 0.7 m/s; this hysteresis prevents the noisy filter heading of a parked vehicle from rotating its footprint.

### C. Prediction

Every confirmed, moving track emits four maneuver hypotheses—keep, brake, drift left and drift right—over a 4 s

horizon at 0.2 s resolution. Their weights depend on two class attributes, erraticness $e$ and willingness to yield $y$:

$$w_{\text{keep}} = 0.88 - 0.50e, \;\; w_{\text{brake}} = 0.10 + 0.18y \quad (3)$$

$$w_{\text{left}} = w_{\text{right}} = \tfrac{1}{2}\max\left(0, 1 - w_{\text{keep}} - w_{\text{brake}}\right) \quad (4)$$

A bus ($e$ = 0.10) places 83% of its weight on *keep*, whereas cattle ($e$ = 0.95) place 41% on *keep* and 49% on lateral drift. Each hypothesis is rolled out with the CTRV model and represented by a chain of discs along the footprint, whose radius grows over the horizon at 0.35 m/s scaled by erraticness and mobility, plus half the track's position uncertainty. Stationary objects receive a single static hypothesis.

### *D. Risk Assessment and Behavior*

Risk assessment measures every quantity *along the road*, in Frenet coordinates ($s$, $d$), rather than along the vehicle's longitudinal axis. It computes the time to collision (TTC) with the nearest in-path object, the minimum clearance to moving and stationary objects, the time for a crossing agent to reach the ego path, and the free lateral space on either side. A six-state machine maps these quantities to planner parameters (Table II). Transitions to a more severe state are immediate; relaxation waits 3–8 replanning cycles.

TABLE II
BEHAVIOR STATES AND PLANNER PARAMETERS

| State | Entered when | Speed cap | Risk tol. |
|---|---|---|---|
| CRUISE | Road clear | $v_{\max}$ | 0.35 |
| FOLLOW | Moving lead within max(12 m, 2$v$) | Lead speed + 0.35 × gap error | 0.35 |
| NUDGE | Clearance < 1.6 m or TTC < 4 s, with room to pass | 0.55 $v_{\max}$ | 0.28 |
| YIELD | Crossing agent reaches path in < 3.5 s | 2.5 m/s | 0.18 |
| CREEP | Confined, or stopped > 1.8 s with > 1 m clearance | 1.8 m/s | 0.21 |
| STOP | TTC < 1.8 s within braking distance, or clearance to a moving object < 0.7 m | 0 | 0.35 |

Two further speed caps share the form

$$v_c = \sqrt{2a_c(g - m)} \quad (5)$$

where $g$ is the gap to the object or goal. An unconfirmed object in the path imposes $a_c$ = 1.5 m/s$^2$ and $m$ = 2.0 m, so a possible ghost slows the vehicle but never stops it; the goal imposes $a_c$ = 0.8 m/s$^2$ and $m$ = 2.5 m, so the vehicle comes to rest at the target. In addition, no candidate path may end within a 5 m standoff behind a stationary object in line with it, so the vehicle stops where it can still steer around. If the vehicle stays stopped for 3 s with an obstacle close ahead and no feasible plan, a reverse maneuver backs it 3 m at 0.8 m/s, steering toward the road heading angled 0.2 rad to the open side.

### *E. Frenet Lattice Planner*

The planner replans every 0.1 s in the Frenet frame of a reference path. Longitudinal motion $s(t)$ is a quartic polynomial from the current state to one of four target speeds over a horizon $T \in \{2.5, 4.0\}$ s. Following [2], lateral motion is a quintic polynomial, but in arc length rather than time:

$$d(\Delta s) = \textstyle\sum_{k=0}^{5} c_k \Delta s^k, \;\; \Delta s = s(t) - s_0 \quad (6)$$

It connects the current offset $d_0$ and slope $d'_0$ to a target offset $d_1$ with zero slope and curvature over the distance that the longitudinal profile actually covers. Under time parameterization a slow vehicle is asked to move sideways before it moves forward, which here demanded a 0.9 m turning radius against a physical minimum of 4.4 m; arc-length parameterization keeps lateral motion proportional to forward progress. Target offsets are sampled every 0.55 m across the permitted width, together with the current offset and the lane center. Because Indian traffic keeps left, the preferred offset on roads at least 5 m wide is the center of the left half; narrower roads are driven along the middle.

Candidates are rejected in cascade on longitudinal acceleration (−4.5 to 2.5 m/s$^2$), path slope $|d'| \le 0.8$, road edges, curvature ≤ 0.45 m$^{-1}$, speed, lateral acceleration ≤ 4.2 m/s$^2$, and the standoff rule. Survivors are checked against every prediction hypothesis, with the ego vehicle covered by three discs inflated by a 0.45 m margin. The risk of a candidate is

$$r = \textstyle\sum_h w_h \max_{a \in h} e^{-t_a/\tau} \quad (7)$$

where $w_h$ is the weight of hypothesis $h$, $t_a$ the first time the ego discs overlap disc $a$ of that hypothesis, and $\tau$ = 2 s. Distant conflicts are discounted because they will be replanned against about 40 more times before they can occur. Candidates whose risk exceeds the current tolerance are discarded, and the rest are ranked by the cost

$$\begin{aligned} J = {} & 60r + 0.008(q_s + q_d) + 12\kappa_{\max}^2 \\ & + 6\,\frac{\max(0, s_0 + v_{\text{cap}}T - s_T)}{T} + 3\lambda(d_1 - d_{\text{lane}})^2 \\ & + 2\left(v_1 - v_{\text{cap}}\right)^2 + 10\max(0, 1.5 - c_{\min})^2 \end{aligned} \quad (8)$$

where $q_s$ and $q_d$ are the mean squared longitudinal and lateral jerk, $\kappa_{\max}$ the peak curvature, $c_{\min}$ the minimum clearance in meters, and $\lambda$ a lane-preference weight, normally 1 but reduced to 0.15 in NUDGE and 0.3 in CREEP so that the vehicle pulls out early around a parked obstacle. If no candidate survives, the planner falls back to the lowest-risk candidate with positive clearance and, failing that, to an emergency stop at −7 m/s$^2$.

### *F. Control and Vehicle Model*

A PI speed controller ($k_p$ = 1.6, $k_i$ = 0.2) with feedforward acceleration tracks the planned speed 28% of the way along the trajectory, with jerk limited to 4 m/s$^3$ when accelerating and 25 m/s$^3$ when braking. Steering follows pure pursuit [12],

$$\delta = \arctan\left(\frac{2L\sin\alpha}{L_d}\right) \quad (9)$$

where $L$ = 2.7 m is the wheelbase, $\alpha$ the bearing to the look-ahead point and $L_d$ = max(3 m, 0.9$v$). The plant is a kinematic bicycle model [13] integrated by fourth-order Runge–Kutta at 20 Hz, with steering limited to ±0.55 rad at 0.6 rad/s and a reverse gear capped at 1.5 m/s.

## V. Implementation

The stack comprises about 10,400 lines in three layers that communicate only through files and a socket protocol. All algorithms are plain functions and structures, with no classes, argument blocks, string arrays or packages, so the same code runs on GNU Octave 7+ and MATLAB R2020b+. Toolbox calls are guarded with hand-written fallbacks, and a start-up script reports the available capability tier so that a run that has silently lost a toolbox is visible.

Profiling showed that Octave's built-in gradient function consumed 77% of a planning cycle; numerically identical column-wise replacements removed it. Batching candidate generation and collision checking into matrix operations reduced planning from 919 ms to 61 ms per cycle in early profiling. Windows power throttling can run a windowless interpreter on efficiency cores at about one third of full speed, so the launcher disables throttling for it.

Every scenario takes a seed, and a fixed seed reproduces a world exactly. A validator rejects random layouts that break road rules, and a batch runner exports each run to JSON, from which Python tools render figures and video, so every picture matches the run that produced its metrics.

A Unity 6 (URP) project supplies a 3D world, rendered sensors and the vehicle. It launches the Octave stack as a server speaking a length-prefixed JSON-plus-binary TCP protocol: Unity streams LiDAR, radar and camera data each tick, the stack returns acceleration and steering, and a C# port of the bicycle model moves the car. With an exact plant, a live run reproduces the offline run to $10^{-13}$ m. An optional Python bridge sits between Unity and the stack, relaying every message and passing each command through a user-editable function, so a user can impose a speed limit or override control while the stack continues to plan from the true vehicle state. Unit tests cover the bicycle model, collision checking, the EKF, footprints, Frenet transforms, Hungarian assignment, perception, polynomials and the reference path, and two closed-loop regression tests run one scripted and one randomized world.

## VI. Experiments and Results

### A. Scenarios

Each scenario targets one failure mode of lane-based planning on Indian roads (Table III). Surrounding agents are reactive: they brake for the ego vehicle and steer around it, but do not plan. The ego vehicle is 4.0 m × 1.8 m with a 2.7 m wheelbase.

TABLE III
Evaluation Scenarios

| Scenario | Situation | $v_{max}$ (m/s) |
|---|---|---|
| Village road | Unmarked road; oncoming auto-rickshaw, cyclist, pedestrian on the verge, parked pushcart | 8 |
| Crossroads | Unsignalized urban intersection; cross traffic that does not yield | 10 |
| Highway merge | Slip-road merge into slow heavy traffic that concedes no gap | 18 |
| Market street | Stalls and pushcarts narrowing the road; pedestrians crossing | 5 |
| Cattle crossing | Open rural road; cattle emerge from behind a parked truck | 12 |

### B. Metrics

Safety is measured against *ground truth*, never against the vehicle's own tracks; otherwise a tracker that lost an agent would report comfortable clearance up to the moment of impact. Clearance is the gap between disc covers of the ego vehicle and each agent, which over-approximate the true rectangles, so it is conservative; a collision is any negative clearance. Comfort is the share of moving time with longitudinal jerk within 4 m/s$^3$ and curvature within 0.30 m$^{-1}$. Latency is the wall-clock time of one replan on a laptop under the Octave interpreter; it indicates relative planner cost, not embedded real-time performance.

### C. Scripted Scenarios

The vehicle reached the goal in all five scripted scenarios with no ground-truth collision (Table IV). Curvature stayed within its limit for 100% of moving time and longitudinal jerk for 85–93%, peak lateral acceleration ranged from 0.59 to 2.52 m/s$^2$, and no replan exceeded the 100 ms budget (worst case 42 ms). The tightest clearances, 0.28 m and 0.40 m, occur where the scenarios force them: cattle stepping out from behind a truck and a market street narrowed by stalls. Times include coming to rest at the target.

TABLE IV
Closed-Loop Results, Scripted Scenarios

| Scenario | Time (s) | Dist. (m) | Mean $v$ (m/s) | Min. clear. (m) | p95 lat. (ms) |
|---|---|---|---|---|---|
| Village road | 43.1 | 190 | 4.41 | 1.05 | 30 |
| Crossroads | 26.1 | 158 | 6.06 | 2.69 | 37 |
| Highway merge | 31.3 | 286 | 9.16 | 3.89 | 30 |
| Market street | 90.5 | 165 | 1.68 | 0.40 | 33 |
| Cattle crossing | 55.9 | 236 | 4.11 | 0.28 | 34 |

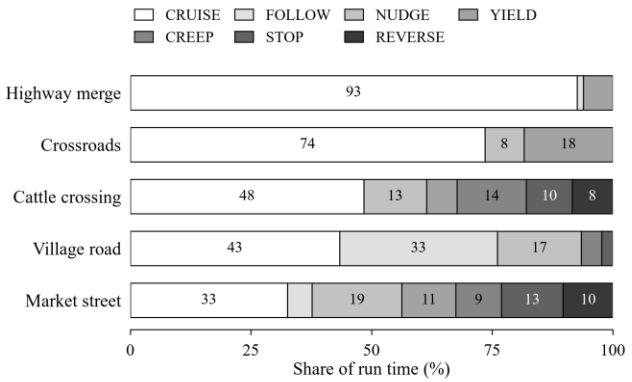



Fig. 2. Share of run time spent in each behavior state, scripted scenarios. Values below 8% are unlabeled.

Fig. 2 shows how the behavior layer adapts to scene density. The highway merge cruises for 93% of its run, whereas the market street and the cattle crossing spend more than half of

their time in NUDGE through REVERSE, and they are the only runs that needed to back out.

### D. Randomized Worlds

Each scenario was also run in three random worlds (seeds 11, 23 and 101) that preserve the scenario's story and add everyday traffic: oncoming vehicles, cycles and carts, vehicles at the curb, and pedestrians and cattle at the roadside. Across these 15 worlds there were no collisions and 14 arrivals. The single failure, village road seed 11, is a timeout without contact (Section VII). Four further worlds (market 376837, village road 13901 and 152526, cattle crossing 849169) all reached the goal without collision.

### E. Co-Simulation With Rendered Sensors

In the Unity co-simulation the stack receives LiDAR, radar and camera data rendered from the 3D scene instead of the analytic sensor models (Table V). Ghosts are detections more than 2.5 m from any road user's center, which includes the ends of an 11 m bus or a truck, so the figure overstates true false positives. Evidence-gated caution is what keeps the 41% ghost rate at the crossroads from causing stops.

TABLE V
LIVE RUNS WITH UNITY-RENDERED SENSORS

| Scenario | Result | Time (s) | LiDAR recall | LiDAR ghosts |
|---|---|---|---|---|
| Village road | Reached | 46.2 | 92% | 0% |
| Crossroads | Reached | 44.7 | 72% | 41% |
| Highway merge | Reached | 31.1 | 100% | 0% |
| Market street | Reached | 75.3 | 97% | 0% |
| Cattle crossing | Timeout (100 s) | — | 95% | 15% |

No live run collided. Driven through the Python bridge with random traffic, village road world 13901 was completed in 55.8 s and crossroads world 23 in 44.3 s, with 95th-percentile step latency of 42–47 ms.

## VII. DISCUSSION AND LIMITATIONS

Three design decisions are supported by the results. First, arc-length lateral parameterization was the most important planner detail: it removed infeasible low-speed sideways motion, and curvature remained within its limit throughout every run. Second, because the behavior layer only reshapes the constraints of one optimizer, the state machine stayed small, every slowdown carries a readable reason, and the machine can be transcribed directly into a Stateflow chart. Third, requiring 0.6 s of tracking and LiDAR corroboration before an object can stop the vehicle allowed it to drive through a 41% LiDAR ghost rate without false stops, while real objects were still confirmed with room to brake.

Four limitations remain. *Perception sets the ceiling:* in village road world 11 a parked bus is tracked with its heading 12° off, so its near corner appears in front of the nose and the vehicle correctly refuses to proceed, waiting about 90 s without contact; the cattle-crossing timeout with Unity sensors has the same character. *Hybrid A* is implemented but disabled:* in isolation it finds escape paths of about 5 m from a boxed-in state, but end to end it raised 95th-percentile latency from about 80 ms to 250–400 ms, cost the crossroads its completion and, planning to a thinner margin than the lattice, produced a contact in a previously clean scenario. *Surrounding traffic is reactive:* agents brake and steer around the ego vehicle but never negotiate, so interaction-aware prediction is untested. *Scenarios are tuned to be passable:* earlier versions were physically impassable in places, so the scripted results show that the stack can succeed, not how often it would on real roads.

Latency was measured in an interpreter on a laptop and is about three times higher if the operating system throttles the process; it reflects relative cost, not real-time capability on vehicle hardware. Mutual deadlocks, in which the ego vehicle and a traffic agent each wait for the other, remain sensitive to timing.

## VIII. CONCLUSION

A lane-free stack built from established components—EKF tracking, multi-hypothesis prediction, a Frenet lattice and pure pursuit—can negotiate unmarked, mixed Indian traffic without collision when those components are adapted to the setting. The adaptations that mattered were arc-length lateral planning, a behavior layer that only shapes the optimizer, footprint-based tracking, evidence-gated stopping and ground-truth evaluation. The stack completed 5 of 5 scripted scenarios and 14 of 15 randomized worlds with zero collisions at 30–37 ms 95th-percentile replanning latency in an interpreter.

Future work will refine the heading of long vehicles as the ego vehicle draws alongside, make Hybrid A* cheap enough and margin-consistent enough to enable as a fallback, transcribe the behavior layer into Simulink and Stateflow, replace the simulated camera with a detector trained on IDD [14], and add interaction-aware traffic agents so that negotiation and mutual deadlock can be studied directly.

## REFERENCES


[1] B. Paden, M. Čáp, S. Z. Yong, D. Yershov, and E. Frazzoli, "A survey of motion planning and control techniques for self-driving urban vehicles," *IEEE Trans. Intell. Veh.*, vol. 1, no. 1, pp. 33–55, Mar. 2016.

[2] M. Werling, J. Ziegler, S. Kammel, and S. Thrun, "Optimal trajectory generation for dynamic street scenarios in a Frenét frame," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2010, pp. 987–993.

[3] M. Pivtoraiko, R. A. Knepper, and A. Kelly, "Differentially constrained mobile robot motion planning in state lattices," *J. Field Robot.*, vol. 26, no. 3, pp. 308–333, 2009.

[4] D. Dolgov, S. Thrun, M. Montemerlo, and J. Diebel, "Path planning for autonomous vehicles in unknown semi-structured environments," *Int. J. Robot. Res.*, vol. 29, no. 5, pp. 485–501, 2010.

[5] M. Montemerlo *et al.*, "Junior: The Stanford entry in the Urban Challenge," *J. Field Robot.*, vol. 25, no. 9, pp. 569–597, 2008.

[6] Y. Bar-Shalom, X. R. Li, and T. Kirubarajan, *Estimation With Applications to Tracking and Navigation*. New York, NY, USA: Wiley, 2001.

[7] R. Schubert, E. Richter, and G. Wanielik, "Comparison and evaluation of advanced motion models for vehicle tracking," in *Proc. 11th Int. Conf. Inf. Fusion*, 2008, pp. 1–6.

[8] H. W. Kuhn, "The Hungarian method for the assignment problem," *Nav. Res. Logist. Q.*, vol. 2, no. 1–2, pp. 83–97, 1955.

[9] S. Blackman and R. Popoli, *Design and Analysis of Modern Tracking Systems*. Norwood, MA, USA: Artech House, 1999.

[10] X. Zhang, W. Xu, C. Dong, and J. M. Dolan, “Efficient L-shape fitting for vehicle detection using laser scanners,” in *Proc. IEEE Intell. Veh. Symp. (IV)*, 2017, pp. 54–59.

[11] S. Lefèvre, D. Vasquez, and C. Laugier, “A survey on motion prediction and risk assessment for intelligent vehicles,” *ROBOMECH J.*, vol. 1, no. 1, pp. 1–14, 2014.

[12] R. C. Coulter, “Implementation of the pure pursuit path tracking algorithm,” Robot. Inst., Carnegie Mellon Univ., Pittsburgh, PA, USA, Tech. Rep. CMU-RI-TR-92-01, 1992.

[13] J. Kong, M. Pfeiffer, G. Schildbach, and F. Borrelli, “Kinematic and dynamic vehicle models for autonomous driving control design,” in *Proc. IEEE Intell. Veh. Symp. (IV)*, 2015, pp. 1094–1099.

[14] G. Varma, A. Subramanian, A. Namboodiri, M. Chandraker, and C. V. Jawahar, “IDD: A dataset for exploring problems of autonomous navigation in unconstrained environments,” in *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, 2019, pp. 1743–1751.